

SEECs: (ro)vibrational Schrödinger Equation solver with Exterior Complex Scaling

As above, so implemented with a B-spline basis.

What ?

Numerically solves the 1D Schrödinger equation for a diatomic molecule,

$$\left[-\frac{1}{2\mu} \frac{d^2}{dR^2} + V(R) + \frac{j(j+1)}{2\mu R^2} \right] \psi(R) = E\psi(R)$$

given a tabulated internuclear potential $V(R)$, a reduced mass μ , and a rotational quantum number j . Returns the energies E , wavefunctions $\psi(R)$ evaluated on a grid, and the rotational constant B_v for each state.

Exterior Complex Scaling (ECS) makes the solutions above the dissociation threshold meaningfully different than (half-)box states. ECS is similar in goal and effect to other methods like Complex Absorbing Potentials (CAPs, AKA optical potentials). CAPs get their name from their method. Adding an imaginary potential to the tail end of the real-valued potential to produce a complex-valued potential beyond some cutoff distance R_0 . ECS differs in that the potential remains real and it is the R-grid that changes past a cutoff distance R_0 : the R-grid past R_0 gets rotated into the complex plane by an angle θ :

$$R \rightarrow z(R) = R_0 + (R - R_0) e^{i\theta}, \quad R > R_0,$$

Waves at $R > R_0$ decay, allowing the continuum to be represented in a finite basis of bound-state wavefunctions (hopefully) without reflection artifacts.

How ?

Uses a basis of clamped B-splines of order k on $[R_{\min}, R_{\max}]$ with `nelem` knot intervals. Boundary conditions are imposed by dropping basis functions at each edge (`ndropl`, `ndropr`). Hamiltonian (H) and overlap (S) matrices are assembled element-by-element with Gauss-Legendre quadrature. The generalized problem $Hc = ES c$ is solved with a real symmetric solver (no ECS) or a complex-symmetric solver (ECS). True bound states in the ECS case will be mostly real, while resonances will have a significant imaginary component associated with the resonance width Γ . The rotational constants

$B_v = \langle v | 1 / \left(2\mu R^2 \right) | v \rangle$ are computed as a quadratic forms on the eigenvectors for each vibrational state..

Units

If called as a library, everything is in atomic units. If the program is called, then the `namelist` parameters can be used to selected unit conversions. The scaling angle `theta` is given in degrees; `R0`, `rmin`, `rmax` are in the same units as the input R grid (`runits_in`).

Building

SEECs builds with `fpm`:

```
fpm build
fpm run < input.namelist
```

LAPACK is required. By default SEECs declares the LAPACK interfaces itself and links a system LAPACK.

Utilities

Some utilities are provided in the `utils` directory: – `multiresonanceplot.jl`: a julia script for plotting ECS energies – `runtemplate`: a `zsh` script for using the ECS template like the one in the `templates` directory and doing several runs

`multiresonanceplot.jl`

Requires the julia programming language. Plots the result of at least one run of SEECs, but only works for complex energies. It expects multiple files and can be called from your shell:

```
julia utils/multiresonanceplot.jl output/ecs*energies*.dat
```

or in a julia script/REPL

```
using Glob
include("utils/multiresonanceplot.jl")
plot_energies(glob("output/ecs*energies.dat"); kwargs...)
```

where `kwargs...` are passed to `Plots.scatter!`

`runtemplate`

Requires the Z shell (`zsh`). This script expects a template `namelist` containing the markers `<<R0>>` and `<<THETA>>` as its positional argument. This can loop over several values for the scaling radius `R0` and angle `THETA`. This is only for the ECS case. For more info, run

```
./utils/runtemplate -h
```

Input

Run the code using `fpm`:

```
fpm run < example/ecs.namelist
```

or directly using the executable

```
path/to/seecs < example/noecs.namelist
```

SEECs reads a Fortran `namelist` via standard input, e.g., `fpm run < input.namelist`, where `input.namelist` contains

```
&control
  k           = 6           ! B-spline order (k = p+1); 6 is a good default
  nelelem     = 200          ! knot intervals; must be >> nwf
  ndropl      = 1           ! drop 1 spline at small-R edge ->  $\psi(R_{min})=0$ 
  ndropr      = 1           ! drop 1 spline at large-R edge ->  $\psi(R_{max})=0$ 
  jrot        = 0           ! rotational quantum number j
  nwf         = 40          ! number of states to return (lowest first)
  nR_wf       = 500         ! points at which to evaluate  $\psi(R)$ 
```

```

redmass = 0.9480647 ! reduced mass (munits_in)

! -- ECS
do_ecs = .true.
R0 = 6.0 ! <-- scaling radius
theta = 15.0 ! <-- scaling angle (degrees)

! rmin = ... ! <-- R-grid override (minimum)
! rmax = ... ! <-- R-grid override (maximum)

! -- files
potential_input_file = "example/V.dat"
energies_output_file = "example/output/ecs_energies.dat"
wfs_output_file = "example/output/ecs_wfs.dat"

! -- input units
runits_in = "bohr" ! R column units
eunits_in = "hartree" ! V column units
munits_in = "amu" ! redmass units

! -- output units
runits_out = "bohr"
eunits_out = "eV"
bunits_out = "invcm"
/

```

Output

Two files are produced for each run of SEECs, given by the following variables:

- wfs_output_file: wavefunctions (solutions)
- energies_output_file: energies and rotational constants of the wavefunctions

Status / limitations

- Just a 1D solver. Nothing multidimensional.
- No build alternative to fpm (waiting on good makefile generation)
- Not really tested yet

License

MIT